

Database Operations

Section Review

Client Payloads

- A **POST** request creates a resource on the server. The server returns a **201 Created** status code if successful.
- POST requests can include a payload/body of data.
- Declare a route handler parameter type as a Pydantic model, and Pydantic will perform validation on the payload.

The Session Object I

- We used our **SessionDep** type to inject a database session into every route handler.
- **SessionDep** uses FastAPI's **Depends** function to invoke the **get_session** function to yield a session.
- The **session** object's **add** method queues up a database operation. Think of it like a rough draft that isn't published until we give the explicit call.

The Session Object II

- The **add** method accepts an object from a model class that connects to a database table. We can modify that object in memory.
- The **commit** method saves the queued changes to the database.
- The **refresh** method updates the Python object with the latest database data.

Queries I

- The **session.exec** method accepts code to build a SQL query statement.
- Pass **select(Model)** to extract all rows from the table corresponding to the model.
- The **exec** method returns an iterable object. Invoke **all** to convert it into a Python list.

Filtering with Where

- The **where** method adds a **WHERE** filtering condition to the SQL statement.
- Reference model attributes with dot syntax and then use logical operators (`==`, `!=`, `>=`, `<`) to apply filtering criteria (**`Room.bedrooms == 3`**).
- Chain multiple **where** method calls to apply AND logic (all conditions must be met).

Filtering by Text

- Prefer to filter database-side. It will be faster compared to Python runtime code.
- The **func** object includes methods for transforming properties (like lowercasing/uppercasing).
- The **contains** method checks if a column value contains the specified string.
- The **ilike** method performs case-insensitive search.

Limits and Offsets

- The **limit** method restricts the query results to a maximum number of rows.
- The **offset** method skips a number of rows before returning results.
- The two constructs are paired together when building pagination.

Ordering Results

- The **order_by** method sorts query results using a model's field. Pass in a column/model attribute as the first argument.
- The default sort order is ascending (smallest to largest for numbers, alphabetical for text, earliest to latest for datetimes).
- The **desc** function sorts the results in descending order.

The Path Function

- FastAPI will identify path parameters automatically based on the parameter name matching the **{id}** in the route. It can also convert the data type.
- The **Path** function uses Pydantic to perform more extensive validation on a path parameter.

PUT vs. PATCH

- A **PUT** request is meant to fully update the resource.
- A **PATCH** request is meant to partially update a resource.
- These are the HTTP standard's rules, but you'll find both HTTP verbs in most codebases.
- The methods can return 200, 201, or 204 No Content.

Looking up a Single Record

- Use **session.get** to fetch a database record by ID. Pass in the SQLAlchemy class, then the ID.
- The method will return either a model instance or **None** if the record is not found.
- Raise a 404 exception if the record does not exist.
- We can also extract the query to a dependency / function that runs before the route handler.

Updating a Record

- The **sqlmodel_update** accepts a model or dictionary and overwrites the fields in the original record.
- This update is limited to code. The database record is not yet affected.
- Use **session.add** with the updated resource and **session.commit** to save the change to the database.
- Use **session.refresh** to refresh the record with DB data.

Multiple Pydantic Models I

- One Pydantic model will soon fail to accommodate all variants of a record (request data, response data, update data etc).
- The recommended solution is to subclass Pydantic models.
- Define a model for the payload to create a record, update a record, the actual DB record, the public record, etc.

Multiple Pydantic Models II

- Try to avoid subclassing Pydantic models that correspond to a DB table (i.e., the ones that have **table=True** in the class parameter list).
- The response shape can remove optionality for fields/attributes like **id** because the value is guaranteed to exist.
- Never send the user sensitive data like passwords.

DELETE Requests

- The **DELETE** HTTP verb is a request to delete a record.
- Invoke **session.delete** and pass in the fetched resource/model object.
- Save the change to the database with the **session.commit** method.
- There is no need to call **refresh**; the record no longer exists in the database.

Response Shape

- Specify a Pydantic model as the return value of a route handler to perform validation/conversion.
- Alternatively, pass a **response_model** parameter to the route decorator method.
- Pydantic will automatically convert the return value to fit the model schema, remove extra properties, etc.
- Pydantic verifies data coming *into* the server and data being sent *back* to the client.

The `model_dump` Method

- The **`model_dump`** method converts a Pydantic model into a Python dictionary.
- The **`exclude_unset`** parameter removes fields that were not explicitly set by the input data (i.e., the **`None`** fallback values that Pydantic provided).
- The **`sqlmodel_update`** method thus receives a smaller dictionary with only the keys corresponding to the fields to update (specified by the client payload).

Dependency Caching

- FastAPI builds a dependency graph. A graph is a data structure where nodes connect to other nodes.
 - Ex. Function A needs dependency B which needs to invoke Function C that needs dependency D
- FastAPI will cache each dependency. It will *not* recreate multiple instances of a dependency like **session**.